Lock-Free Synchronization Paradigms in Real-Time Audio Graphs

Bill Westlake

July 30, 2026

Abstract

This paper explores the mathematical and systemic constraints of hard real-time execution environments, specifically within digital audio workstation (DAW) audio processing graphs. We demonstrate why standard operating system synchronization primitives, such as mutexes and semaphores, inevitably violate real-time safety via priority inversion. We then formalize lock-free architectures—specifically Single-Producer/Single-Consumer (SPSC) ring buffers and atomic state swapping—as necessary paradigms for maintaining deterministic execution bounds in high-performance C++ DSP applications.

1 Introduction

In modern real-time audio systems, the processing graph must supply a continuous stream of audio samples to the Digital-to-Analog Converter (DAC) under strict temporal constraints. Failure to meet these deadlines results in buffer underruns, physically manifesting as audible artifacts (clicks and pops).

The fundamental challenge arises when the audio thread must communicate with the user interface (UI) thread or perform dynamic graph topology alterations. This paper bridges theoretical systems engineering with practical C++ implementations, illustrating how to circumvent priority inversion and maintain lock-free determinism.

2 The Mathematical Constraints of the Audio Callback

3 Priority Inversion and the Failure of Mutexes

4 Lock-Free Ring Buffers

5 Atomic State Swapping and Double-Buffering

6 Conclusion